

Video RAG Retrieval Project

<https://github.com/aiyashwary/Video-RAG-Retrieval>

OVERVIEW

A semantic video search system that extracts frames, generates scene captions using ViT-GPT2 SentenceTransformers and OpenCLIP into a dual FAISS index, enabling natural language queries to retrieve video scenes with timestamps.

IMPACT

Built a Visual RAG system that makes unstructured video content semantically searchable: a text query can retrieve video scenes with timestamps by matching against dual text and image embedding indexes via cosine similarity.

TECHNICAL WALKTHROUGH

I built a Visual Retrieval-Augmented Generation (RAG) system that retrieves relevant video segments by combining vision models and vector similarity search.

Searching videos using text is hard because video content is unstructured. Traditional keyword search finds what is actually happening inside a video. My project solves this by converting both video scenes and queries into embeddings and retrieving the most relevant video segments.

- i) Scene understanding using vision models
- ii) Vector storage for text and image embeddings
- iii) Similarity-based retrieval using VDMS

iv) Video → Scene Description + Frames → Embeddings → Vector DB → Retrieval → Video Clip

The input to the system is one or more videos. Each video is processed to extract: Scene-level descriptions, key video frames along with metadata like frame timestamps using open-source vision models like Video-LLaMA.

Vision-language models are used to understand the video content. These models analyze video frames and generate language descriptions that summarize what's happening in each scene. For example, a scene description might be 'A person walking into a store and picking up an item'.

To support efficient retrieval, two types of embeddings are stored in separate vector stores:

- i) Text embeddings: Model: SentenceTransformer – all-MiniLM-L6-V2 / L12-V2. Used for scene descriptions. This model converts textual descriptions into dense vectors that capture semantic meaning.
- ii) Image embeddings: Model: OpenCLIP – ViT-g-14. Used for video frames. This helps capture visual features, actions, and context directly from frames.

Both text and image embeddings are stored in VDMS, which allows fast vector similarity search. The system maintains separate vector stores for text and image embeddings but they are linked using video metadata.

- i) Stored metadata includes: Video name, Scene ID, Frame number, Timestamp

When a user enters a text prompt: Prompt is converted into a text embedding. Vector similarity search is performed against the text embeddings and (optionally) image embeddings. The system retrieves the most relevant scene descriptions and timestamps.

Scene description and exact time range to jump to in the video. Text embeddings capture semantic context, while image embeddings capture visual features. Using both improves retrieval accuracy because some queries are better answered by visual similarity and others by textual context.

-
- i) 'a red car' → image embeddings
 - ii) 'a tense conversation' → text embeddings
 - iii) Efficient vector search using cosine similarity
 - iv) Decoupling embeddings for scalability
 - v) Model choice based on speed vs accuracy
 - vi) Frame sampling using FPS to avoid redundancy

This approach makes video content searchable, explainable, and scalable, and it can be extended to support real-time or enterprise video search.

A Visual RAG system enables semantic search over videos. It uses vision models to generate scene captions from video frames, converts them into text and image embeddings using SentenceTransformers and OpenAI embeddings, and retrieves relevant video segments based on user queries.

KEY DETAILS

1. Extracts frames at a configurable FPS and generates natural-language scene captions using a vision model.
2. Computes text embeddings with SentenceTransformers (MiniLM) and image embeddings with OpenAI embeddings, then indexes them.
3. Stores embeddings and metadata in two parallel FAISS indexes supporting independent text and image search.

-
4. Query interface accepts a natural language string and returns top-k matching scenes with frame-level timestamps.
 5. Modular architecture (preprocessing, embeddings, storage, retrieval) designed for extension to other datasets and models.
 6. FAISS indexes use IndexFlatIP with L2-normalized vectors for cosine-similarity search; OpenAI CLIP embeddings on LAION-2B with automatic GPU/CPU selection.
 7. Includes a pytest suite and a self-contained demo that synthesizes a 30-frame test video with an index-and-query pipeline without needing a real video file.